INF384 · CASO MODELO PARA EL EXAMEN 1

Caso: Bodega Online, venta de abarrotes por aplicación

Bodega Online vende abarrotes por aplicación móvil con entrega el mismo día en Lima. Tiene un servicio de pedidos (Python, contenedores) y un servicio de inventario que mantiene otro equipo. Seis desarrolladores trabajan con ramas de funcionalidad que duran unas dos semanas. Dos personas de QA validan a mano en un ambiente de certificación compartido. El área de seguridad revisa manualmente cada salida y responde en un plazo de tres días. El despliegue lo ejecuta una persona de operaciones con un procedimiento escrito.

Value stream map de un cambio típico (horas calendario)

Paso	**PT**	**LT**	**%C&A**
Refinamiento	3 h	48 h	80%
Desarrollo	12 h	40 h	90%
Revisión de código	2 h	30 h	85%
QA manual en ambiente compartido	8 h	96 h	60%
Revisión manual de seguridad	2 h	72 h	90%
Despliegue manual	3 h	26 h	75%

Datos del último mes (4 semanas): 6 despliegues a producción; 1 exigió rollback; ese incidente tardó 6 h en restaurarse; tuvimos 10 commits con una mediana de tiempo de 11 días para llegar a producción.

Pipeline actual (resumen del archivo del repositorio)

```yaml
on: push                          # todas las ramas
jobs:
  build:
    steps:
      - run: pip install -r requirements.txt   # sin versiones fijas
      - run: docker build -t registro/cf-pedidos:${{ github.run_number }} .
  pruebas:
    steps:
      - run: pytest || true
  seguridad:
    steps:
      - run: <analisis-de-dependencias>        # publica un reporte
  deploy-prd:
    needs: build
    steps:
      - run: docker build -t registro/cf-pedidos:prd . && docker push registro/cf-
pedidos:prd
      - run: <desplegar> registro/cf-pedidos:prd
```

Dockerfile usado por aplicación:

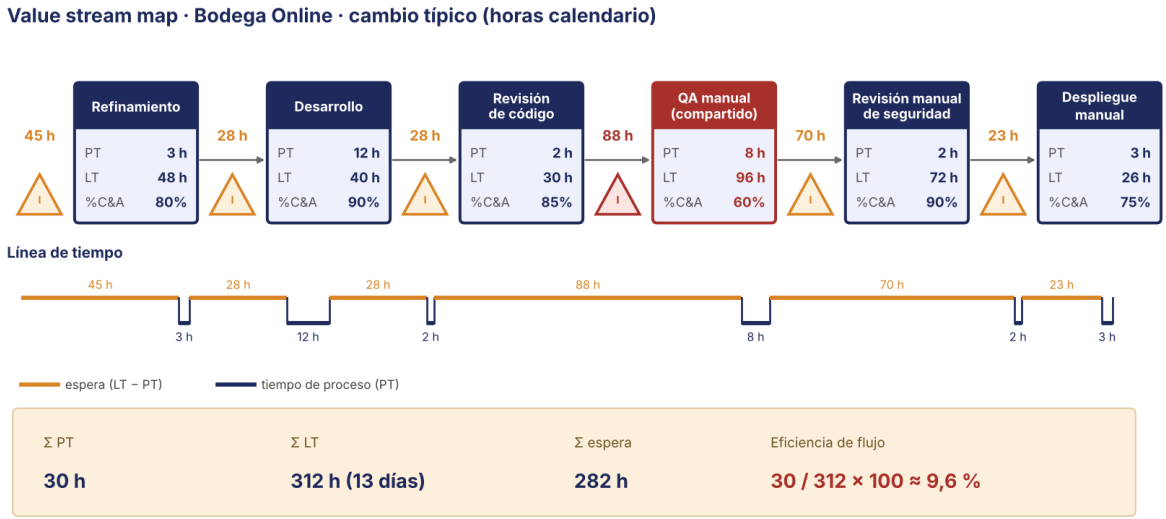
```
FROM python:latest
WORKDIR /app
COPY . .
RUN apt-get update && apt-get install -y curl git
RUN pip install -r requirements.txt
RUN pip install pytest
ENV DB_PASSWORD=CampoFresco2026!
ENV AMBIENTE=produccion
EXPOSE 8000
CMD ["python", "app.py"]
```

Próximo hito. En cinco semanas llega el Cyber Wow, cuando Bodega Online espera cinco veces su tráfico normal. Para esa fecha necesita reemplazar la columna direccion de los pedidos por calle, numero y distrito, para asignar repartidores por zona. El equipo propone una migración que elimina direccion y agrega las tres columnas nuevas, desplegada con reemplazo progresivo (rolling) a las 9:00am del primer día del Cyber Wow.

Preguntas modelo · tipo Parte A (obligatoria)

A1 (2 puntos). Diagrama el VSM y calcula la eficiencia de flujo del VSM. Indique

dónde intervendría primero y justifique con los datos de la tabla, incluido el %C&A.



ETAPA	PT: Trabajo (h)	Espera (h)	%C&A	LT: Subtotal (h)
Refinamiento	3 h	45 h	80%	48 h
Desarrollo	12 h	28 h	90%	40 h
Revisión de código	2 h	28 h	85%	30 h
QA manual en ambiente compartido	8 h	88 h	60%	96 h
Revisión manual de seguridad	2 h	70 h	90%	72 h
Despliegue manual	3 h	23 h	75%	26 h
TOTAL	**30 h**	**282**	**-**	**312**

Tiempo de trabajo efectivo: 30 h

Tiempo de espera: 282 h

Tiempo total del flujo: 312 h

Eficiencia de flujo del VSM = ΣPT / ΣLT × 100 = 30 h / 312 h × 100 = 9,62 % ≈ 9,6 %. Es muy baja: cerca del 90 % del lead time (282 h de 312 h) es espera y solo 30 h es trabajo efectivo.

La primera intervención debe hacerse en QA manual. Es el paso con mayor LT (96 h), mayor espera pura (88 h, el 28 % del lead time total) y además el %C&A más bajo (60 %). Por tanto, no solo concentra cola, sino también retrabajo: 4 de cada 10 entregas no llegan completas y correctas y vuelven a desarrollo. Se interviene con pruebas automatizadas en el pipeline y ambientes efímeros que reemplacen la validación manual en el ambiente compartido, y con un quality gate antes de QA que suba el %C&A, dejando el ambiente compartido solo para lo que requiere una aplicación desplegada. La revisión manual de seguridad (70 h de espera) es el segundo foco válido: se reemplaza por controles en el pipeline (caso B-2).

A2 (2 puntos). Mide las métricas DORA y en base a ello clasifica a Bodega Online. Indique qué

métrica considera más crítica antes del Cyber Wow y qué relación tiene con el VSM.

Métrica	Dato	Nivel DORA
Frecuencia de despliegue	6 despliegues en 4 semanas (≈ 1,5 por semana)	Medio (cerca del límite con Alto)
Lead time para cambios	Mediana de 11 días (≈ 264 h)	Medio (entre una semana y un mes)
Tasa de fallas en cambios	1 / 6 ≈ 16,7 % (1 rollback en 6 despliegues)	Medio (15 – 45 %)
Tiempo de restauración del servicio	6 h	Alto (menos de un día)

Clasificación: Bodega Online queda en nivel Medio. Tres de las cuatro métricas están en Medio y solo el tiempo de restauración llega a Alto. Las ramas de dos semanas, el QA y la seguridad manuales y el despliegue manual mantienen lotes grandes y un lead time de más de una semana.

Métrica más crítica antes del Cyber Wow: la tasa de fallas en cambios (1 de cada 6 despliegues falló), junto con el tiempo de restauración. Con cinco veces el tráfico normal, un despliegue fallido afecta a muchos más clientes y 6 h de recuperación significan ventas perdidas; además la restauración depende de una persona y de un procedimiento manual, y el cambio de esquema previsto es justamente el tipo de cambio de mayor riesgo.

Relación con el VSM: el lead time DORA (mediana de 11 días) equivale al LT del VSM desde Desarrollo hasta Despliegue (40 + 30 + 96 + 72 + 26 = 264 h = 11 días). El VSM explica por qué es tan largo y dónde intervenir (QA manual y revisión de seguridad concentran 158 h de espera); DORA es la medición continua y automática que dice si esas intervenciones están mejorando el flujo.

A3 (2 puntos). Sobre el pipeline y el Dockerfile actuales:

- **a) (1 punto)** Identifique dos defectos del pipeline. Para cada uno, explique qué problema concreto puede causar en producción y cómo lo corregiría.

- Defecto 1:
  - Descripción: las dependencias se instalan sin versiones fijas (pip install -r requirements.txt, sin lock file).
  - Problema concreto que puede causar en prod: el mismo commit produce artefactos distintos según el día; puede entrar una versión nueva de una librería (con vulnerabilidades o cambios incompatibles) que nadie probó y, ante un incidente, no se puede reproducir ni saber qué versión estaba en producción.
  - Cómo lo corregiría: versionar un lock file (versiones exactas y huellas, incluidas las dependencias transitivas), instalar estrictamente desde él en el pipeline y tratar cada actualización de versión como un cambio explícito que se revisa y se prueba.
- Defecto 2:
  - Descripción: pytest se ejecuta con || true, por lo que las pruebas nunca detienen el pipeline.
  - Problema concreto que puede causar en prod: se construye, publica y despliega código con pruebas fallidas; la calidad no está asegurada y un

defecto llega a producción, donde en el Cyber Wow afectaría a cinco veces más usuarios.
    - Cómo lo corregiría: quitar el || true para que un fallo de pytest devuelva error y detenga el pipeline, y hacer que la publicación de la imagen y deploy-prd dependan del job de pruebas (needs), de modo que no exista artefacto si las pruebas fallan.
- Otros defectos del pipeline: el job de seguridad solo publica un reporte y no detiene nada (falta una compuerta con umbrales); on: push se ejecuta en todas las ramas (debería desplegarse solo desde main); la imagen se etiqueta con el número de ejecución y no con el SHA del commit; deploy-prd vuelve a construir la imagen en lugar de promover el mismo artefacto ya probado, identificado por su digest; y deploy-prd no depende de las pruebas ni de seguridad.

● **b) (1 punto)** Identifique dos defectos del DockerFile, cuál es el impacto y como lo corregirías.

- Defecto 1:
    - Descripción: la imagen base no tiene versión fija (FROM python:latest).
    - Impacto: dos builds del mismo commit pueden producir imágenes distintas; una actualización de la imagen base (con cambios incompatibles o vulnerabilidades) llega a producción sin haberse probado y se pierde la reproducibilidad.
    - Cómo lo corregiría: fijar una versión específica y una base ligera (por ejemplo, una variante slim de una versión concreta de Python).
- Defecto 2:
    - Descripción: COPY . . copia todo el contexto de construcción a la imagen (incluidos .git, archivos .env y pruebas).
    - Impacto: puede filtrar credenciales y metadatos del repositorio dentro de la imagen, aumenta la superficie de ataque y el tamaño, e invalida la caché de capas en cada cambio.
    - Cómo lo corregiría: copiar primero solo el manifiesto de dependencias y el lock file, instalar, y luego copiar únicamente el código necesario; agregar un archivo .dockerignore.
- Además: (1) ENV DB_PASSWORD deja la contraseña hardcodeada en una capa visible con docker history; debe inyectarse al desplegar; (2) ENV AMBIENTE=produccion fija un valor por ambiente dentro del artefacto, lo que obliga a construir por ambiente y destruye la promoción del mismo digest; (3) el proceso corre como root: declarar un USER sin privilegios; (4) instala curl, git y pytest en la imagen final: usar construcción en dos etapas (build y runtime); (5) falta un HEALTHCHECK; (6) pip install sin lock file.

A4 (2 puntos). El equipo propone eliminar la columna direccion y desplegar el cambio con reemplazo progresivo (rolling) a las 9:00 del primer día del Cyber Wow.

● **a) (0.5 puntos)** Explique qué puede fallar durante el despliegue con esta propuesta y por qué ya no se podría volver a la versión anterior.

Con el reemplazo progresivo (rolling) conviven durante un tiempo contenedores con la versión vieja y con la nueva. Si la migración elimina direccion, las instancias viejas que todavía atienden pedidos fallan al leer o escribir una columna que ya no existe, justo a las 9:00 del primer día del Cyber Wow con cinco veces el tráfico. Además, ya no se podría volver a la versión anterior: revertir el código no restituye la columna ni los datos de dirección eliminados, de modo que el artefacto anterior es incompatible con el esquema actual y solo quedaría corregir hacia adelante, con el servicio degradado.

● **b) (1 punto)** Describa en pasos cómo haría el cambio de columnas sin perder la

posibilidad de volver atrás, e indique cuándo lo desplegaría.

Se aplica el patrón expand-contract (expandir y contraer) en despliegues separados, manteniendo siempre la versión anterior compatible con el esquema:

1) Expandir: agregar las columnas calle, numero y distrito sin tocar direccion (la versión en ejecución sigue funcionando y se puede revertir).

2) Escribir en ambas estructuras y migrar datos: la versión nueva guarda direccion y también calle, numero y distrito; se migran los pedidos históricos.

3) Trasladar la lectura: la lectura y la asignación de repartidores por zona pasan a las columnas nuevas, controladas con un feature flag (o un canary); si algo falla se apaga el flag sin desplegar.

4) Contraer: direccion se elimina después del Cyber Wow, en un despliegue propio, cuando ninguna versión en ejecución la lee.

Estado de las columnas: antes: direccion; tras los pasos 1 a 3: direccion + calle + numero + distrito; tras estabilizarse (después del Cyber Wow): calle + numero + distrito.

Cuándo: los pasos 1 a 3 se despliegan semanas antes del Cyber Wow (quedan 5 semanas), para dar tiempo a estabilizar y validar los datos migrados. El día del evento no se despliega ningún cambio de esquema; a lo sumo se activa el feature flag. El paso 4 se hace después del evento.

- **c) (0.5 puntos)** Defina qué métrica y qué valor revisaría durante el Cyber Wow para decidir volver a la versión anterior.

Métrica: tasa de errores de las peticiones (5xx) al crear y consultar pedidos, junto con la proporción de pedidos sin distrito asignado (datos nuevos vacíos), observadas en una ventana de observación durante el Cyber Wow. Valor para revertir, declarado antes del evento: por ejemplo, tasa de errores mayor a 1 % durante 5 minutos consecutivos o pedidos sin distrito asignado por encima de lo esperado; en ese caso se revierte sin consultar (se apaga el feature flag o se vuelve al digest anterior).

Preguntas modelo · tipo Parte B (opcionales)

En el examen se eligen tres de seis casos. Aquí hay dos casos de muestra.

B-2 · Bodega Online: reemplazar la revisión manual de seguridad

Bodega Online quiere reemplazar la revisión manual de tres días por controles en el pipeline. En la primera ejecución aparecen estos hallazgos:

#	Control	Hallazgo
1	Secretos	Contraseña de la base de datos en un commit de hace un año, borrada después
2	Composición	CVE crítica en una dependencia declarada; existe versión corregida
3	Análisis estático	80 hallazgos preexistentes; ninguno nuevo en este pull request

4	Análisis de IaC	Bucket de almacenamiento declarado con acceso público

a) (2 puntos) Para cada hallazgo, decida si detiene el pipeline, se registra o requiere excepción, y justifique.

#	Control	Decisión	Justificación
1	Secretos	Detiene el pipeline; rotar la contraseña y limpiar el historial. No admite excepción.	Aunque la línea se borró, el valor sigue en el historial de git: borrarla no resuelve nada. Una credencial nunca se exceptúa: se retira, se limpia del historial y se rota.
2	Composición (SCA)	Detiene; subir a la versión corregida.	Severidad crítica, en una dependencia declarada (alcanzable) y con corrección disponible: no hay motivo para dejarla pasar.
3	Análisis estático	Se registra e inventaría (no detiene).	Son hallazgos heredados y el pull request no introduce ninguno nuevo. El gate detiene lo nuevo e inventaría lo heredado, que se planifica y corrige después.
4	Análisis de IaC	Detiene; declarar el bucket como privado.	Exposición pública declarada en la infraestructura: es un hallazgo crítico de seguridad que debe corregirse antes de desplegar.

Una excepción solo se usaría para un hallazgo que no se pueda corregir de inmediato y se acepte de forma temporal (nunca una credencial): se registra con identificador del hallazgo, razón, responsable y fecha de vencimiento, y al vencer el pipeline vuelve a detenerse. Ninguno de los cuatro hallazgos requiere excepción.

b) (1 punto) Indique qué papel conserva el área de seguridad cuando ya no revisa cada salida. **c) (1 punto)** Relacione cada control con una práctica de NIST SP 800-218.

b) El área de seguridad deja de revisar cada salida y pasa a definir y gobernar la política como código: fija los umbrales (qué detiene y qué solo se registra), aprueba y da seguimiento a las excepciones con responsable y vencimiento, revisa los cambios de alto riesgo y audita los resultados y el número de excepciones vigentes. Deja de ser una etapa de espera de 72 horas: su trabajo ocurre antes (configurar las reglas) y después (auditar y responder a incidentes), no en cada despliegue.

c) Relación de cada control con una práctica del NIST SP 800-218 (SSDF):

- Detección de secretos → PW.7: revisar o analizar el código legible para encontrar vulnerabilidades (complementaria: PS.1, proteger el código contra acceso no autorizado).
- Análisis de composición (SCA) → PW.4: reutilizar software existente y verificar los componentes de terceros.
- Análisis estático (SAST) → PW.7: revisar o analizar el código legible para encontrar vulnerabilidades.
- Análisis de IaC → PW.7: analizar la declaración de infraestructura (como código) en busca de vulnerabilidades (complementaria: PW.9, configurar el software para que sea seguro por defecto).